

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM

TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

KHOA CÔNG NGHỆ THÔNG TIN



Hồ Nguyễn An Khang – 52200020

HỆ THỐNG CHẤM BÀI TRỰC TUYẾN - TDTUOJ

KHÓA LUẬN TỐT NGHIỆP

Kỹ thuật phần mềm

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM

TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

KHOA CÔNG NGHỆ THÔNG TIN



Hồ Nguyễn An Khang – 52200020

HỆ THỐNG CHẤM BÀI TRỰC TUYẾN - TDTUOJ

KHÓA LUẬN TỐT NGHIỆP

Kỹ thuật phần mềm

Người hướng dẫn:

ThS. Doãn Xuân Thanh

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Em xin chân thành cảm ơn quý thầy cô Khoa Công nghệ Thông tin, Trường Đại học Tôn Đức Thắng đã tận tình giảng dạy và trang bị cho em những kiến thức nền tảng vững chắc trong suốt quá trình học tập. Đặc biệt, em xin gửi lời cảm ơn sâu sắc đến giảng viên hướng dẫn **ThS. Doãn Xuân Thanh** đã dành thời gian định hướng, góp ý và hỗ trợ em trong từng giai đoạn thực hiện đề tài này. Bên cạnh đó, em xin cảm ơn gia đình và bạn bè đã luôn động viên, tạo điều kiện để em có thể hoàn thành tốt dự án.

TP. Hồ Chí Minh, ngày ... tháng ... năm 20...

Tác giả

(Ký tên và ghi rõ họ tên)

LỜI CAM ĐOAN

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Tôi xin cam đoan đây là công trình nghiên cứu của riêng tôi và được sự hướng dẫn khoa học của **ThS. Doãn Xuân Thanh**. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong Dự án còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào tôi xin hoàn toàn chịu trách nhiệm về nội dung Dự án của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày ... tháng ... năm 20...

Tác giả

(Ký tên và ghi rõ họ tên)

HỆ THỐNG CHẤM BÀI TRỰC TUYẾN - TDTUOJ

TÓM TẮT

Dự án này trình bày quá trình nghiên cứu, thiết kế và triển khai ban đầu của Hệ thống Chấm bài Trực tuyến TDTUOJ – một nền tảng được xây dựng nhằm tự động hóa quy trình chấm bài lập trình và hỗ trợ tổ chức thi cử cho sinh viên Khoa Công nghệ Thông tin, Trường Đại học Tôn Đức Thắng.

Hệ thống được thiết kế để phục vụ ba nhóm người dùng chính gồm sinh viên, giảng viên và quản trị viên, với các chức năng cốt lõi bao gồm nộp và chấm bài tự động cho nhiều ngôn ngữ lập trình, quản lý cuộc thi và bài toán, và phân tích hỗ trợ học tập thông minh bằng Large Language Model – Mô hình ngôn ngữ lớn (LLM). Kiến trúc hệ thống được xây dựng theo mô hình hai lớp tách biệt rõ ràng giữa backend và frontend, tích hợp với ba dịch vụ bên ngoài gồm Judging Service (Judge0), LLM và AI Parser Service.

Đến thời điểm báo cáo giữa kỳ, hệ thống đã hoàn thành phần lớn các chức năng cốt lõi. Phía backend được xây dựng bằng Java với Spring Boot, đã hiện thực hóa đầy đủ các Application Programming Interface (API) quản lý người dùng, quản lý bài toán và luồng xử lý bài nộp tích hợp Judge0. Phía frontend được xây dựng bằng React, đã hoàn thiện giao diện xem chi tiết bài toán theo slug, quản lý người dùng, tính năng nộp bài và hỗ trợ học tập bằng LLM.

ONLINE JUDGE SYSTEM - TDTUOJ

ABSTRACT

This project presents the research, design, and initial implementation of the TDTUOJ Online Judge System – a platform built to automate programming submission grading and support examination organization for students of the Faculty of Information Technology at Ton Duc Thang University.

The system is designed to serve three main user groups: students, lecturers, and administrators, with core features including automatic submission judging for multiple programming languages via Judge0, contest and problem management, and AI-powered learning support via LLM integration. The system architecture follows a clear two-layer separation between a Java Spring Boot backend and a React frontend, integrated with three external services: Judging Service (Judge0), LLM, and AI Parser Service.

As of the midterm report, the majority of core features have been implemented. The backend provides complete API coverage for user management, problem management, and Judge0-integrated submission processing. The frontend has completed the problem detail view by slug, user management interface, submission functionality, and LLM-powered hint and code explanation features.

MỤC LỤC

LỜI CẢM ƠN

LỜI CAM ĐOAN

TÓM TẮT

ABSTRACT

DANH MỤC HÌNH VẼ **iv**

DANH MỤC BẢNG BIỂU **v**

DANH MỤC CÁC CHỮ VIẾT TẮT **vi**

1	Mở đầu và tổng quan đề tài	1
1.1	Lý do chọn đề tài	1
1.2	Khảo sát	1
1.3	Mục đích nghiên cứu	2
1.4	Phạm vi nghiên cứu	2
1.5	Ý nghĩa nghiên cứu	3
2	Kiến trúc nền tảng	4
2.1	Vòng đời phát triển phần mềm	4
2.2	Các công nghệ sử dụng	5
2.2.1	Backend – Java và Spring Boot	5
2.2.2	Cơ sở dữ liệu – PostgreSQL	6
2.2.3	Containerization – Docker	6
2.2.4	Hàng đợi và Bộ nhớ đệm – Redis	7
2.2.5	Dịch vụ chấm bài – Judge0	7

2.2.6	Frontend – React	8
2.2.7	Lưu trữ file – AWS S3	8
2.2.8	Tích hợp LLM	9
3	Phân tích hệ thống	11
3.1	Thu thập yêu cầu	11
3.2	Xử lý nghiệp vụ	12
3.2.1	Các bên liên quan và thực thể nghiệp vụ	12
3.2.2	Các tác nhân hệ thống	13
3.3	Yêu cầu chức năng (Functional Requirements)	16
3.3.1	Tổng quan Use Case	16
3.3.2	Các Use Case chính	23
3.4	Yêu cầu phi chức năng (Non-functional Requirements)	23
4	Thiết kế hệ thống	24
4.1	Thiết kế kiến trúc	24
4.2	Thiết kế cơ sở dữ liệu / ERD	24
4.2.1	Nhóm bảng quản lý người dùng	24
4.2.2	Nhóm bảng quản lý bài toán	24
4.2.3	Nhóm bảng quản lý cuộc thi	24
4.2.4	Nhóm bảng quản lý bài nộp	24
4.3	Thiết kế lớp	24
4.4	Thiết kế giao diện	24
4.4.1	Trang chủ	24
4.4.2	Trang danh sách bài toán	24
4.4.3	Trang chi tiết bài toán và nộp bài	24
4.4.4	Trang cuộc thi và bảng xếp hạng	24
4.4.5	Trang quản trị	24
5	Cài đặt và kiểm thử	25

6 Kết luận	26
TÀI LIỆU THAM KHẢO	27

DANH MỤC HÌNH VẼ

3.1	Sơ đồ Use Case tổng quan của hệ thống TDTUOJ	22
4.1	Kiến trúc tổng quan của hệ thống TDTUOJ	24
4.2	Sơ đồ thực thể – quan hệ của hệ thống TDTUOJ	24
4.3	Sơ đồ lớp của hệ thống TDTUOJ	24

DANH MỤC BẢNG BIỂU

3.1	Các tác nhân của hệ thống TDTUOJ	14
3.2	Danh sách các Use Case của hệ thống TDTUOJ	17

DANH MỤC CÁC CHỮ VIẾT TẮT

AC	Accepted
AI	Artificial Intelligence – Trí tuệ nhân tạo
API	Application Programming Interface
CE	Compilation Error
CNTT	Công nghệ Thông tin
CSS	Cascading Style Sheets
ERD	Entity-Relationship Diagram – Sơ đồ thực thể quan hệ
HTTP	Hypertext Transfer Protocol
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model – Mô hình ngôn ngữ lớn
MLE	Memory Limit Exceeded
MVC	Model-View-Controller
OJ	Online Judge
PDF	Portable Document Format
PK	Primary Key – Khóa chính
FK	Foreign Key – Khóa ngoại
RE	Runtime Error

REST	Representational State Transfer
TDTU	Trường Đại học Tôn Đức Thắng
TLE	Time Limit Exceeded
UI	User Interface
UX	User Experience
WA	Wrong Answer

CHƯƠNG 1. MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI

1.1 Lý do chọn đề tài

Hiện nay, việc tổ chức các kỳ thi lập trình và rèn luyện kỹ năng giải thuật cho sinh viên tại Trường Đại học Tôn Đức Thắng vẫn còn nhiều hạn chế về mặt công cụ hỗ trợ. Chấm bài thủ công tiêu tốn nhiều thời gian và công sức của giảng viên, đồng thời khó đảm bảo tính nhất quán khi số lượng sinh viên ngày càng tăng. Bên cạnh đó, sinh viên thiếu một môi trường luyện tập chủ động với phản hồi tức thì để tự nâng cao kỹ năng lập trình ngoài giờ học.

Để giải quyết những vấn đề này, đề tài đề xuất xây dựng Hệ thống Chấm bài Trực tuyến TDTUOJ – một nền tảng tự động hóa toàn bộ quy trình chấm bài, hỗ trợ tổ chức thi cử quy mô lớn và tạo ra môi trường luyện tập lập trình chuyên nghiệp trong khuôn khổ học thuật của nhà trường.

1.2 Khảo sát

Trước khi xây dựng TDTUOJ, nhóm đã tiến hành khảo sát một số hệ thống Online Judge (OJ) phổ biến hiện có nhằm đánh giá ưu nhược điểm và xác định khoảng trống mà hệ thống cần lấp đầy.

Codeforces là một trong những nền tảng thi lập trình lớn nhất thế giới, nổi bật với hệ thống rating Elo uy tín, kho bài toán phong phú và hạ tầng chịu tải cao. Tuy nhiên, Codeforces hoàn toàn là nền tảng công cộng và không cung cấp cơ chế quản lý lớp học, phân quyền theo tổ chức hay tích hợp với quy trình đào tạo nội bộ của một trường đại học cụ thể.

LeetCode tập trung vào luyện tập kỹ năng phỏng vấn với giao diện hiện đại và tính năng giải thích lời giải chi tiết. Mặc dù vậy, nền tảng này không hỗ trợ tổ chức thi cử theo lịch học, không có cơ chế phân quyền giảng viên – sinh viên và không thể tùy biến theo nhu cầu học thuật nội bộ.

VNOJ (Vietnam Online Judge) là hệ thống OJ mã nguồn mở phổ biến tại Việt

Nam, hỗ trợ tiếng Việt và có cộng đồng người dùng trong nước lớn. Dù VNOJ phù hợp cho luyện tập cá nhân và một số kỳ thi học sinh sinh viên, hệ thống vẫn thiếu các tính năng nâng cao như hỗ trợ học tập bằng LLM, nhập đề bài tự động từ Portable Document Format (PDF) và cơ chế quản lý tổ chức linh hoạt theo mô hình đại học.

Qua khảo sát, nhóm nhận thấy các hệ thống hiện có đều không được thiết kế để phục vụ nhu cầu đặc thù của một đơn vị đào tạo cụ thể: quản lý phân quyền theo vai trò giảng viên và sinh viên, tích hợp Artificial Intelligence – Trí tuệ nhân tạo (AI) hỗ trợ học tập trực tiếp trong luồng nộp bài, và khả năng vận hành độc lập trong môi trường nội bộ của nhà trường. Đây chính là khoảng trống mà TDTUOJ được xây dựng để lấp đầy.

1.3 Mục đích nghiên cứu

TDTUOJ được xây dựng nhằm đạt một tập hợp các mục tiêu cụ thể và có tính ứng dụng cao. Về mặt kỹ thuật, hệ thống hỗ trợ đa ngôn ngữ lập trình, cho phép người dùng nộp bài viết bằng C, C++, Java và Python, đáp ứng nhu cầu đa dạng của sinh viên ở nhiều trình độ và môn học khác nhau. Song song với đó, hệ thống cung cấp đầy đủ công cụ để quản lý cuộc thi và bài tập, bao gồm tổ chức contest, quản lý danh sách bài toán, chấm điểm tự động và xếp hạng người tham gia theo thời gian thực.

Ngoài các chức năng cốt lõi, hệ thống tích hợp một số tính năng nâng cao phục vụ học tập. Tính năng phân tích mã nguồn bằng LLM cho phép giải thích code, gợi ý hướng giải và phân tích độ phức tạp thuật toán ngay trên giao diện – giúp sinh viên hiểu rõ hơn về hiệu suất và logic của giải pháp mình đề xuất. Ngoài ra, hệ thống tích hợp AI Parser Service để tự động đọc đề bài từ file PDF và chuyển đổi thành dữ liệu có cấu trúc, giúp giảng viên tiết kiệm đáng kể thời gian soạn thảo.

1.4 Phạm vi nghiên cứu

Đề tài tập trung xây dựng một hệ thống hoàn chỉnh phục vụ sinh viên và giảng viên Khoa Công nghệ Thông tin, Trường Đại học Tôn Đức Thắng. Hệ thống quản lý ba vai trò với phân quyền rõ ràng: User (người dùng thông thường), Contest Creator

(người tổ chức và quản lý cuộc thi), và Admin (quản trị viên với toàn quyền điều hành). Về mặt kiến trúc, phạm vi đề tài bao gồm phát triển một backend Representational State Transfer (REST)ful API bằng Java Spring Boot, một giao diện người dùng bằng React, và tích hợp ba dịch vụ bên ngoài: Judging Service (Judge0), LLM và AI Parser Service.

Những nội dung nằm ngoài phạm vi của đề tài bao gồm: phát triển ứng dụng di động, tích hợp hệ thống quản lý học tập (LMS) hiện có của nhà trường, xây dựng hạ tầng chấm bài tùy chỉnh thay thế Judge0, và hỗ trợ các ngôn ngữ lập trình ngoài bốn ngôn ngữ đã nêu ở giai đoạn đầu.

1.5 Ý nghĩa nghiên cứu

TDTUOJ không chỉ là một công cụ hỗ trợ giảng dạy đơn thuần mà là một nền tảng tổng thể có khả năng tạo ra tác động tích cực và lâu dài cho cộng đồng học thuật của nhà trường.

Về ý nghĩa thực tiễn, với sinh viên, hệ thống tạo ra môi trường luyện tập lập trình chuyên nghiệp với phản hồi tức thì, phân tích thuật toán chuyên sâu và hỗ trợ học tập thông minh từ LLM – những yếu tố giúp người học phát triển tư duy giải quyết vấn đề một cách chủ động và hiệu quả. Với giảng viên, hệ thống giải phóng nguồn lực khỏi các công việc lặp đi lặp lại như chấm bài thủ công và soạn thảo đề bài, từ đó tập trung nhiều hơn vào việc thiết kế nội dung học thuật chất lượng cao.

Về ý nghĩa khoa học, đề tài đặt ra và giải quyết một số bài toán kỹ thuật có giá trị nghiên cứu: xây dựng luồng xử lý bài nộp bất đồng bộ có khả năng mở rộng theo chiều ngang, tích hợp LLM vào quy trình đánh giá học tập, và áp dụng chiến lược cache phân tầng để phục vụ bảng xếp hạng thời gian thực. Ở tầm nhìn rộng hơn, TDTUOJ góp phần xây dựng văn hóa lập trình thi đấu tại Trường Đại học Tôn Đức Thắng (TDTU), đồng thời thể hiện năng lực nghiên cứu và phát triển phần mềm thực tiễn của sinh viên Khoa Công nghệ Thông tin (CNTT).

CHƯƠNG 2. KIẾN TRÚC NỀN TẢNG

2.1 Vòng đời phát triển phần mềm

Để phù hợp với bối cảnh phát triển cá nhân, yêu cầu hệ thống có thể thay đổi trong quá trình triển khai và thời gian thực hiện có hạn của một đề tài tốt nghiệp, dự án TDTUOJ áp dụng phương pháp phát triển theo hướng Agile kết hợp với mô hình quản lý công việc bằng Kanban board trên nền tảng Trello.

Thay vì chia dự án thành các sprint có độ dài cố định như Scrum, phương pháp tiếp cận được chọn là luồng công việc liên tục (continuous flow): các hạng mục công việc được đưa vào thực hiện tuần tự dựa trên mức độ ưu tiên, và tiến độ được điều chỉnh linh hoạt theo từng giai đoạn phát triển thực tế. Đây là lựa chọn thực tế và hiệu quả cho một dự án cá nhân, nơi việc tổ chức hợp sprint hay điều phối nhóm không phát sinh và mọi quyết định kỹ thuật được đưa ra trực tiếp dựa trên kết quả của từng tác vụ đã hoàn thành.

Kanban board trong Trello được tổ chức thành bốn cột có ý nghĩa rõ ràng và liên kết trực tiếp với vòng đời của từng hạng mục công việc. Cột **Backlog** tập hợp toàn bộ các tính năng, yêu cầu kỹ thuật và tác vụ đã được xác định nhưng chưa được lên lịch thực hiện – đây là nguồn dữ liệu đầu vào chính để lập kế hoạch công việc theo từng giai đoạn. Cột **To Do** chứa các hạng mục đã được phân tích, ưu tiên hóa và sẵn sàng để bắt đầu thực hiện trong thời gian sắp tới. Cột **Doing** phản ánh các tác vụ đang được triển khai tích cực, với nguyên tắc giới hạn số lượng công việc đồng thời để tránh phân tán tập trung. Cột **Done** ghi nhận tất cả hạng mục đã hoàn thành, đã kiểm thử và được tích hợp vào nhánh chính – đây cũng là cơ sở để đánh giá tiến độ tổng thể của dự án.

Quá trình phát triển được chia thành ba giai đoạn lớn, phản ánh thứ tự ưu tiên kỹ thuật từ nền tảng đến tính năng nâng cao. Giai đoạn đầu tập trung vào việc xây dựng nền tảng kiến trúc và các tính năng cốt lõi: thiết lập cấu trúc dự án Spring Boot, cấu hình PostgreSQL và Docker Compose, hiện thực hóa hệ thống xác thực và phân

quyền bằng JSON Web Token (JWT), và hoàn thiện luồng nộp bài tích hợp Judge0. Giai đoạn thứ hai mở rộng sang quản lý cuộc thi và bảng xếp hạng: xây dựng các API quản lý contest, tích hợp Redis cho submission queue và leaderboard cache, và phát triển giao diện frontend tương ứng bằng React. Giai đoạn thứ ba bổ sung các tính năng nâng cao: tích hợp LLM cho hỗ trợ học tập, xây dựng AI Parser Service để nhập đề bài từ PDF, và hoàn thiện giao diện quản trị.

Mỗi hạng mục trong Backlog được mô tả đủ chi tiết để có thể ước lượng thời gian thực hiện và xác định các phụ thuộc kỹ thuật trước khi chuyển sang cột To Do. Cách tổ chức này giúp đảm bảo luôn có sự rõ ràng về công việc tiếp theo cần làm, đồng thời tạo ra cơ sở hữu hình để đánh giá tiến độ so với kế hoạch tổng thể của đề tài.

2.2 Các công nghệ sử dụng

Hệ thống TDTUOJ được xây dựng trên một tập hợp các công nghệ được lựa chọn có chủ đích, đảm bảo sự cân bằng giữa hiệu năng, khả năng bảo trì và tính phù hợp với quy mô của một dự án học thuật. Phần này trình bày lần lượt từng công nghệ, lý do lựa chọn và vai trò cụ thể của nó trong kiến trúc tổng thể.

2.2.1 Backend – Java và Spring Boot

Backend của TDTUOJ được xây dựng bằng ngôn ngữ Java với framework Spring Boot. Spring Boot là một framework phát triển ứng dụng doanh nghiệp trưởng thành, cung cấp cơ chế cấu hình tự động (auto-configuration) giúp rút ngắn đáng kể thời gian thiết lập dự án so với Spring thuần túy, đồng thời tích hợp sẵn một hệ sinh thái phong phú gồm Spring Security, Spring Data Java Persistence API (JPA) và Spring Web.

Kiến trúc backend tuân theo mô hình Model-View-Controller (MVC) phân lớp rõ ràng. Lớp Controller tiếp nhận các yêu cầu Hypertext Transfer Protocol (HTTP) và định tuyến đến lớp Service tương ứng. Lớp Service chứa toàn bộ logic nghiệp vụ, thực hiện xác thực dữ liệu và điều phối các thao tác cần thiết. Lớp Repository, được

hiện thực hóa thông qua Spring Data JPA, chịu trách nhiệm tương tác với cơ sở dữ liệu thông qua các entity và query có kiểu dữ liệu tường minh.

Bảo mật được xử lý thông qua Spring Security kết hợp với JWT. Khi người dùng đăng nhập thành công, hệ thống tạo ra một access token có thời hạn ngắn và một refresh token có thời hạn dài hơn. Mọi yêu cầu API sau đó cần đính kèm access token trong header Authorization; Spring Security sẽ xác thực token, giải mã thông tin người dùng và kiểm tra quyền hạn trước khi cho phép yêu cầu tiếp tục xử lý. Cơ chế này đảm bảo hệ thống hoàn toàn không lưu trạng thái phiên (stateless) ở phía server, phù hợp với kiến trúc RESTful và dễ dàng mở rộng theo chiều ngang khi cần.

2.2.2 Cơ sở dữ liệu – PostgreSQL

PostgreSQL được chọn làm hệ quản trị cơ sở dữ liệu chính của hệ thống. So với MySQL, PostgreSQL cung cấp hỗ trợ tốt hơn cho các kiểu dữ liệu phức tạp như JSONB, kiểu mảng và các biểu thức chỉ mục nâng cao – những tính năng phát huy giá trị trong ngữ cảnh lưu trữ cấu hình bài toán và kết quả chấm bài đa dạng. Ngoài ra, PostgreSQL có cơ chế xử lý giao dịch đồng thời (MVCC) mạnh mẽ, đảm bảo tính toàn vẹn dữ liệu trong các tình huống nhiều bài nộp được xử lý đồng thời.

Kết nối giữa Spring Boot và PostgreSQL được quản lý thông qua HikariCP – connection pool mặc định của Spring Boot, nổi bật với hiệu năng cao và overhead thấp. Các tham số pool như kích thước tối đa, thời gian chờ kết nối và thời gian nhàn rỗi được cấu hình phù hợp với tải dự kiến của hệ thống, đảm bảo việc tái sử dụng kết nối hiệu quả và tránh tình trạng cạn kiệt kết nối trong giờ cao điểm.

2.2.3 Containerization – Docker

Toàn bộ hệ thống TDTUOJ được đóng gói và triển khai bằng Docker, với Docker Compose đóng vai trò điều phối các service chạy cùng nhau trong một môi trường thống nhất. Cách tiếp cận này mang lại lợi thế quan trọng là tính nhất quán giữa môi trường phát triển và môi trường triển khai: mọi thành phần từ backend, database đến judging service đều chạy trong container có cấu hình xác định, loại bỏ hoàn toàn

các vấn đề phụ thuộc môi trường.

Hạng mục đặc biệt cần lưu ý là Judge0 – dịch vụ chấm bài mã nguồn mở – được tự triển khai (self-hosted) thông qua Docker Compose thay vì sử dụng API công cộng có phí. Judge0 yêu cầu quyền truy cập đặc quyền (privileged container) để có thể tạo môi trường sandbox cô lập khi thực thi mã nguồn người dùng, và Docker cung cấp đúng cấp độ kiểm soát cần thiết cho yêu cầu này. Việc tự triển khai Judge0 cũng đảm bảo hệ thống hoạt động hoàn toàn nội bộ mà không phụ thuộc vào dịch vụ bên ngoài, phù hợp với yêu cầu vận hành độc lập của một nền tảng học thuật.

2.2.4 Hàng đợi và Bộ nhớ đệm – Redis

Redis được tích hợp vào hệ thống để phục vụ hai mục đích có tính chất khác nhau nhưng đều đòi hỏi tốc độ truy cập cao.

Mục đích thứ nhất là hàng đợi bài nộp (submission queue). Khi người dùng nộp bài, yêu cầu không được gửi đồng bộ trực tiếp đến Judge0 mà được đặt vào hàng đợi Redis. Một worker riêng biệt lấy yêu cầu từ hàng đợi và gửi đến Judge0 theo thứ tự, sau đó cập nhật kết quả vào cơ sở dữ liệu khi có phản hồi. Cơ chế này tách biệt luồng tiếp nhận yêu cầu khỏi luồng xử lý chấm bài, giúp hệ thống phản hồi người dùng ngay lập tức và xử lý bài nộp bất đồng bộ một cách có kiểm soát, đặc biệt quan trọng trong các thời điểm nhiều người nộp bài đồng thời.

Mục đích thứ hai là cache bảng xếp hạng (leaderboard cache). Dữ liệu bảng xếp hạng của một cuộc thi cần được truy vấn thường xuyên và có tính chất thay đổi theo từng lần nộp bài thành công – việc tính toán lại từ cơ sở dữ liệu quan hệ mỗi lần có yêu cầu sẽ gây tải không cần thiết. Redis lưu trữ bảng xếp hạng đã tính sẵn và chỉ làm mới khi có bài nộp mới ảnh hưởng đến thứ hạng, giảm đáng kể số lượng truy vấn nặng đến PostgreSQL.

2.2.5 Dịch vụ chấm bài – Judge0

Judge0 là dịch vụ thực thi mã nguồn mã nguồn mở, cung cấp khả năng biên dịch và chạy mã trong môi trường sandbox cô lập với giới hạn thời gian và bộ nhớ

được kiểm soát chặt chẽ. Judge0 hỗ trợ hơn 60 ngôn ngữ lập trình, trong đó TDTUOJ hiện khai thác bốn ngôn ngữ chính gồm C, C++, Java và Python.

Luồng tích hợp hoạt động như sau: sau khi một yêu cầu chấm bài được lấy ra từ hàng đợi Redis, hệ thống gửi mã nguồn cùng với các test case tương ứng đến API của Judge0 thông qua giao thức HTTP. Judge0 thực thi mã trong container sandbox, so sánh đầu ra với đáp án mẫu và trả về kết quả bao gồm trạng thái (Accepted (AC), Wrong Answer (WA), Time Limit Exceeded (TLE), Memory Limit Exceeded (MLE), Runtime Error (RE), Compilation Error (CE)) cùng với thời gian thực thi và lượng bộ nhớ sử dụng. Hệ thống sau đó cập nhật bản ghi submission trong PostgreSQL và thông báo kết quả đến frontend thông qua cơ chế polling hoặc WebSocket.

2.2.6 Frontend – React

Giao diện người dùng của TDTUOJ được xây dựng bằng React – thư viện JavaScript phổ biến cho phép xây dựng giao diện theo mô hình component và quản lý trạng thái ứng dụng một cách có cấu trúc. React cho phép xây dựng Single-Page Application trong đó việc điều hướng giữa các trang diễn ra hoàn toàn ở phía client mà không cần tải lại trang, mang lại trải nghiệm người dùng mượt mà và phản hồi nhanh.

Giao tiếp giữa frontend và backend được thực hiện thông qua các RESTful API HTTP. Toàn bộ dữ liệu trao đổi sử dụng định dạng JavaScript Object Notation (JSON), và access token JWT được đính kèm trong header của mỗi yêu cầu cần xác thực. Cấu trúc component của ứng dụng được tổ chức theo từng domain chức năng – người dùng, bài toán, cuộc thi, bài nộp – giúp việc bảo trì và mở rộng tính năng trở nên thuận tiện.

2.2.7 Lưu trữ file – AWS S3

Hệ thống sử dụng AWS S3 (Simple Storage Service) để lưu trữ các tệp nội dung bài toán thay vì nhúng trực tiếp vào cơ sở dữ liệu quan hệ. Cụ thể, S3 được dùng để lưu trữ hai loại nội dung chính: tệp Markdown (.md) chứa đề bài của từng bài toán,

và các tệp văn bản (.txt) chứa dữ liệu đầu vào và đầu ra kỳ vọng của từng test case.

Việc tách nội dung đề bài và test case ra khỏi PostgreSQL mang lại một số lợi thế thực tế. Đề bài dạng Markdown có thể chứa các đoạn mô tả dài, công thức toán học và ví dụ minh họa – những nội dung có kích thước biến động lớn và không cần lập chỉ mục tìm kiếm ở cấp cơ sở dữ liệu. Tương tự, dữ liệu test case có thể bao gồm đầu vào nhiều dòng với kích thước đáng kể tùy theo bài toán; lưu trữ chúng trên S3 giúp tránh làm phình to bảng dữ liệu và đơn giản hóa quy trình thêm hoặc cập nhật test case mà không cần thay đổi schema.

Chiến lược lưu trữ trên S3 được tổ chức theo cấu trúc key có phân cấp rõ ràng, phân tách theo loại nội dung và định danh bài toán, giúp việc truy xuất và quản lý vòng đời của tệp được thực hiện một cách hệ thống. Backend chỉ lưu trữ đường dẫn S3 tham chiếu trong PostgreSQL; khi cần phục vụ nội dung cho người dùng hoặc cho Judge0, hệ thống đọc trực tiếp nội dung tệp từ S3 thông qua API nội bộ. Về định hướng phát triển, hệ thống có thể mở rộng để hỗ trợ lưu trữ tệp PDF đề bài gốc khi tính năng nhập đề từ PDF được tích hợp đầy đủ trong tương lai.

2.2.8 Tích hợp LLM

TDTUOJ tích hợp LLM như một lớp hỗ trợ học tập thông minh, được thiết kế để bổ sung vào luồng giải bài của sinh viên mà không thay thế quá trình tư duy độc lập. Tích hợp này hoạt động theo mô hình proxy: backend đóng vai trò trung gian giữa frontend và LLM API bên ngoài, cho phép kiểm soát nội dung yêu cầu, giới hạn tần suất gọi và ghi nhật ký sử dụng.

Hệ thống cung cấp hai nhóm tính năng hỗ trợ học tập chính. Nhóm tính năng thứ nhất phục vụ sinh viên trực tiếp trong quá trình giải bài: tính năng **gợi ý hướng giải** (hint generation) cung cấp một gợi ý có định hướng mà không tiết lộ lời giải hoàn chỉnh, giúp sinh viên tự tìm ra bước tiếp theo; tính năng **phân tích độ phức tạp** (complexity analysis) nhận vào đoạn mã nguồn và bài toán tương ứng, sau đó trả về đánh giá độ phức tạp thời gian và không gian kèm theo giải thích chi tiết. Nhóm tính năng thứ hai phục vụ giảng viên trong quá trình soạn thảo nội dung: tính năng **AI**

Parser cho phép tải lên tệp PDF đề bài, LLM sẽ phân tích cấu trúc tài liệu và tự động điền các trường thông tin bài toán bao gồm tiêu đề, mô tả, định dạng đầu vào – đầu ra và ví dụ mẫu, giúp giảng viên tiết kiệm đáng kể thời gian nhập liệu thủ công.

Tất cả các yêu cầu đến LLM đều được xây dựng với prompt có cấu trúc rõ ràng ở phía backend, trong đó ngữ cảnh bài toán (đề bài, giới hạn, ví dụ) được đưa vào system prompt để đảm bảo phản hồi của LLM luôn liên quan và chính xác theo từng bài toán cụ thể.

CHƯƠNG 3. PHÂN TÍCH HỆ THỐNG

3.1 Thu thập yêu cầu

Thu thập yêu cầu là một giai đoạn then chốt trong quy trình phát triển phần mềm và quản lý dự án. Đây là quá trình thu thập, ghi chép và phân tích một cách có hệ thống các nhu cầu, kỳ vọng và ràng buộc của các bên liên quan đối với một hệ thống cụ thể. Quá trình này đóng vai trò nền tảng cho toàn bộ dự án, vì chất lượng của sản phẩm cuối cùng phụ thuộc trực tiếp vào mức độ đầy đủ và chính xác của các yêu cầu được xác định từ giai đoạn đầu. Một quy trình thu thập yêu cầu hiệu quả thường bao gồm nhiều kỹ thuật như phỏng vấn, khảo sát, bảng câu hỏi, quan sát người dùng và phân tích tài liệu nhằm thu thập góc nhìn toàn diện từ các nhóm người dùng khác nhau. Thông tin thu thập được sau đó được tổ chức thành tài liệu đặc tả yêu cầu rõ ràng, súc tích, đóng vai trò định hướng cho các nhà phát triển, nhà thiết kế và quản lý dự án trong suốt vòng đời phát triển.

Trong đề tài này, hai kỹ thuật chính được áp dụng để thu thập yêu cầu của hệ thống là phỏng vấn và khảo sát. Đối với kỹ thuật phỏng vấn, nhằm có được cái nhìn đầy đủ từ cả hai phía – người học và người giảng dạy – tác giả đã tiến hành phỏng vấn trực tiếp với một số sinh viên đang theo học tại Khoa Công nghệ Thông tin và một giảng viên phụ trách các học phần lập trình. Để đảm bảo tính hệ thống và đầy đủ của thông tin, các câu hỏi phỏng vấn được chuẩn bị trước theo cấu trúc có định hướng, tập trung vào những khó khăn thực tế trong quá trình học tập, rèn luyện kỹ năng lập trình và tham gia các kỳ thi lập trình nội bộ. Các buổi phỏng vấn được thực hiện theo hình thức trao đổi trực tiếp, cho phép tác giả đặt thêm câu hỏi làm rõ khi cần thiết, từ đó nắm bắt được những nhu cầu tiềm ẩn mà người dùng có thể không diễn đạt trực tiếp trong phiếu khảo sát.

Bên cạnh phỏng vấn, việc nói chuyện trực tiếp với một số cá nhân tuy cung cấp thông tin có chiều sâu nhưng chưa đủ để phản ánh ý kiến của toàn bộ cộng đồng sinh viên. Vì vậy, kỹ thuật khảo sát trực tuyến cũng được áp dụng nhằm nâng cao độ

tin cậy và tính đại diện của dữ liệu thu thập được. Bảng khảo sát được thiết kế với các câu hỏi rõ ràng, dễ hiểu, hướng đến các vấn đề phổ biến mà sinh viên gặp phải khi luyện tập lập trình và thi cử, đồng thời khảo sát mức độ quan tâm đối với các tính năng hỗ trợ học tập thông minh như gợi ý thuật toán hay phân tích mã nguồn bằng AI. Khảo sát được phân phối đến sinh viên từ nhiều khóa học và năm học khác nhau nhằm đảm bảo sự đa dạng trong phản hồi.

Kết luận: Sau khi tổng hợp kết quả từ phỏng vấn và khảo sát, tác giả nhận thấy rằng phần lớn sinh viên gặp khó khăn trong việc tìm kiếm môi trường luyện tập lập trình có phản hồi tức thì, đặc biệt ngoài giờ học chính khóa. Các giảng viên cũng bày tỏ nhu cầu cần một công cụ chấm bài tự động, giảm thiểu thời gian xử lý thủ công và hỗ trợ tổ chức thi cử quy mô lớn một cách hiệu quả. Đây là cơ sở thực tiễn trực tiếp cho việc xác định các yêu cầu chức năng và phi chức năng của hệ thống TDTUOJ được trình bày trong các phần tiếp theo.

3.2 Xử lý nghiệp vụ

Dựa trên kết quả thu thập yêu cầu từ các buổi phỏng vấn và khảo sát, phần này trình bày phân tích chi tiết về các quy trình nghiệp vụ trọng tâm và các bên liên quan chính trong hệ thống.

3.2.1 Các bên liên quan và thực thể nghiệp vụ

Bài toán (Problem)

Bài toán là đối tượng trung tâm của hệ thống. Mỗi bài toán được sở hữu và quản lý bởi người dùng có vai trò Contest Creator hoặc Admin. Quy trình quản lý bài toán có sự phân cấp rõ ràng: bài toán có thể được tạo mới, chỉnh sửa và xóa bởi người có thẩm quyền, đồng thời phải được cấu hình đầy đủ về giới hạn thời gian, giới hạn bộ nhớ, danh sách test case và đề bài trước khi đưa vào sử dụng trong cuộc thi hoặc công khai cho sinh viên luyện tập. Khi bài toán được công khai, sinh viên có thể tìm kiếm, xem chi tiết và nộp bài giải trực tiếp trên hệ thống.

Bài nộp (Submission)

Bài nộp được tạo ra khi sinh viên gửi lời giải cho một bài toán. Mỗi bài nộp ghi lại đầy đủ thông tin gồm mã nguồn, ngôn ngữ lập trình được chọn, thời điểm nộp và kết quả chấm bài. Kết quả của bài nộp phản ánh mức độ chính xác và hiệu quả của lời giải thông qua một trong các trạng thái: AC, WA, TLE, MLE, RE hoặc CE. Dữ liệu bài nộp được lưu trữ lâu dài và là cơ sở để tính toán bảng xếp hạng trong các cuộc thi, cũng như thống kê tiến độ học tập của từng sinh viên.

Cuộc thi (Contest)

Cuộc thi là đơn vị tổ chức thi cử trong hệ thống, được tạo và quản lý bởi người dùng có vai trò Contest Creator hoặc Admin. Một cuộc thi bao gồm tập hợp các bài toán được chọn lọc, thời gian bắt đầu và kết thúc xác định, cũng như danh sách người tham gia đã đăng ký. Trong suốt thời gian diễn ra cuộc thi, bảng xếp hạng được cập nhật theo thời gian thực dựa trên kết quả bài nộp của từng thí sinh. Sau khi cuộc thi kết thúc, kết quả được lưu lại và có thể được truy xuất để phục vụ mục đích thống kê và báo cáo.

Kết quả chấm bài (Judge Result)

Kết quả chấm bài được tạo ra bởi Judge0 sau khi xử lý một bài nộp. Đây là đầu ra của quá trình thực thi mã nguồn trong môi trường sandbox có kiểm soát, bao gồm trạng thái kết quả, thời gian thực thi thực tế và lượng bộ nhớ tiêu thụ trên từng test case. Kết quả này được hệ thống tiếp nhận qua cơ chế bất đồng bộ, cập nhật vào cơ sở dữ liệu và phản hồi đến người dùng thông qua giao diện frontend.

3.2.2 Các tác nhân hệ thống

Hệ thống TDTUOJ được thiết kế để phục vụ ba nhóm tác nhân người dùng với phân quyền rõ ràng, cùng với ba hệ thống bên ngoài đóng vai trò dịch vụ hỗ trợ. Bảng 3.1 mô tả chi tiết từng tác nhân và vai trò tương ứng trong hệ thống.

Bảng 3.1: Các tác nhân của hệ thống TDTUOJ

STT	Tác nhân	Mô tả
1	User (Người dùng)	Đây là tác nhân cơ bản nhất trong hệ thống, đại diện cho sinh viên hoặc người học lập trình nói chung. Người dùng có thể đăng ký tài khoản, đăng nhập, tìm kiếm và xem chi tiết bài toán, nộp bài giải và xem lịch sử bài nộp của bản thân. Người dùng cũng có thể đăng ký tham gia các cuộc thi đang diễn ra, xem bảng xếp hạng và sử dụng các tính năng hỗ trợ học tập do LLM cung cấp như gợi ý hướng giải và phân tích độ phức tạp thuật toán.
2	Contest Creator (Người tổ chức)	Tác nhân này chịu trách nhiệm tổ chức và quản lý các hoạt động thi cử và nội dung bài toán trong hệ thống. Contest Creator có thể tạo mới, chỉnh sửa và xóa bài toán, quản lý danh sách test case, tổ chức và điều hành các cuộc thi bao gồm cấu hình thời gian, danh sách bài toán và người tham gia. Ngoài ra, Contest Creator có thể sử dụng tính năng AI Parser để nhập đề bài tự động từ tệp PDF, giúp tiết kiệm thời gian soạn thảo nội dung một cách đáng kể.

(tiếp theo Bảng 3.1)

STT	Tác nhân	Mô tả
3	Admin (Quản trị viên)	Đây là tác nhân có quyền hạn cao nhất trong hệ thống, chịu trách nhiệm vận hành và quản trị toàn diện nền tảng. Admin có toàn quyền đối với tài khoản người dùng bao gồm tạo mới, chỉnh sửa, vô hiệu hóa hoặc thay đổi vai trò của bất kỳ tài khoản nào trong hệ thống. Ngoài ra, Admin kế thừa toàn bộ quyền hạn của Contest Creator và có thể can thiệp vào bất kỳ bài toán hay cuộc thi nào khi cần thiết.
4	Judge0	Đây là dịch vụ chấm bài bên ngoài được tự triển khai (self-hosted) thông qua Docker. Judge0 đóng vai trò thực thi mã nguồn trong môi trường sandbox cô lập, so sánh đầu ra với đáp án mẫu và trả về kết quả chấm bài cho hệ thống. Judge0 không tương tác trực tiếp với người dùng mà hoạt động hoàn toàn như một dịch vụ nền được gọi thông qua API nội bộ.

(tiếp theo Bảng 3.1)

STT	Tác nhân	Mô tả
5	LLM Service	Đây là dịch vụ LLM bên ngoài được tích hợp thông qua API. Backend đóng vai trò proxy trung gian, tiếp nhận yêu cầu từ frontend, xây dựng prompt có cấu trúc với ngữ cảnh bài toán phù hợp, gửi đến LLM và chuyển tiếp phản hồi về cho người dùng. Dịch vụ này hỗ trợ các tính năng gợi ý hướng giải, phân tích độ phức tạp thuật toán và giải thích mã nguồn.
6	AI Parser Service	Đây là dịch vụ phân tích tài liệu PDF sử dụng LLM làm nền tảng. Khi Contest Creator tải lên một tệp đề bài dưới định dạng PDF, AI Parser Service phân tích cấu trúc tài liệu và trích xuất tự động các trường thông tin cần thiết của bài toán bao gồm tiêu đề, mô tả, định dạng đầu vào – đầu ra và các ví dụ mẫu, trả về dữ liệu có cấu trúc để hệ thống tự động điền vào biểu mẫu tạo bài toán.

3.3 Yêu cầu chức năng (Functional Requirements)

3.3.1 Tổng quan Use Case

Hệ thống TDTUOJ được phân tích theo mô hình Use Case với ba nhóm tác nhân người dùng chính (User, Contest Creator, Admin) và ba hệ thống bên ngoài (Judge0, LLM Service, AI Parser Service). Sơ đồ Use Case tổng quan được thể hiện trong Hình 3.1.

Bảng 3.2: Danh sách các Use Case của hệ thống TDTUOJ

Mã UC	Tên Use Case	Mô tả
Nhóm 1: Xác thực và tài khoản		
UC01	Đăng ký tài khoản	Người dùng tạo tài khoản mới bằng cách cung cấp thông tin cần thiết. Hệ thống xác thực dữ liệu đầu vào và tạo tài khoản với vai trò User mặc định.
UC02	Đăng nhập	Người dùng đăng nhập vào hệ thống bằng tên đăng nhập và mật khẩu. Hệ thống xác thực thông tin đăng nhập và cấp phát cặp access token – refresh token theo cơ chế JWT.
UC03	Đăng xuất	Người dùng đăng xuất khỏi hệ thống. Hệ thống thu hồi token phiên làm việc hiện tại.
UC04	Đổi mật khẩu	Người dùng thay đổi mật khẩu tài khoản của mình sau khi xác thực mật khẩu cũ.
UC05	Cập nhật thông tin cá nhân	Người dùng cập nhật thông tin hồ sơ cá nhân bao gồm tên hiển thị và ảnh đại diện.
Nhóm 2: Quản lý bài toán		
UC06	Xem danh sách bài toán	Người dùng xem danh sách các bài toán đang công khai trong hệ thống. Hệ thống hỗ trợ tìm kiếm theo tên, lọc theo thẻ tag và mức độ khó.
UC07	Xem chi tiết bài toán	Người dùng xem toàn bộ thông tin của một bài toán bao gồm đề bài, định dạng đầu vào – đầu ra, giới hạn thời gian, giới hạn bộ nhớ và các ví dụ mẫu.

(tiếp theo Bảng 3.2)

Mã UC	Tên Use Case	Mô tả
UC08	Tạo bài toán	Contest Creator hoặc Admin tạo một bài toán mới, nhập đầy đủ thông tin mô tả, cấu hình giới hạn và thêm các test case tương ứng.
UC09	Chỉnh sửa bài toán	Contest Creator hoặc Admin cập nhật thông tin của một bài toán đã tồn tại bao gồm nội dung đề bài, cấu hình và danh sách test case.
UC10	Xóa bài toán	Contest Creator hoặc Admin xóa một bài toán khỏi hệ thống. Thao tác này chỉ được phép nếu bài toán không đang được sử dụng trong bất kỳ cuộc thi nào đang diễn ra.
Nhóm 3: Nộp bài và xem kết quả		
UC11	Nộp bài giải	Người dùng nộp mã nguồn lời giải cho một bài toán với ngôn ngữ lập trình được chọn. Hệ thống đưa bài nộp vào hàng đợi Redis và xử lý bất đồng bộ thông qua Judge0.
UC12	Xem kết quả bài nộp	Người dùng xem kết quả chấm bài của một lần nộp cụ thể bao gồm trạng thái, thời gian thực thi và lượng bộ nhớ sử dụng trên từng test case.
UC13	Xem lịch sử bài nộp	Người dùng xem toàn bộ lịch sử các lần nộp bài của bản thân theo từng bài toán hoặc toàn hệ thống, có thể lọc theo trạng thái kết quả và ngôn ngữ lập trình.

(tiếp theo Bảng 3.2)

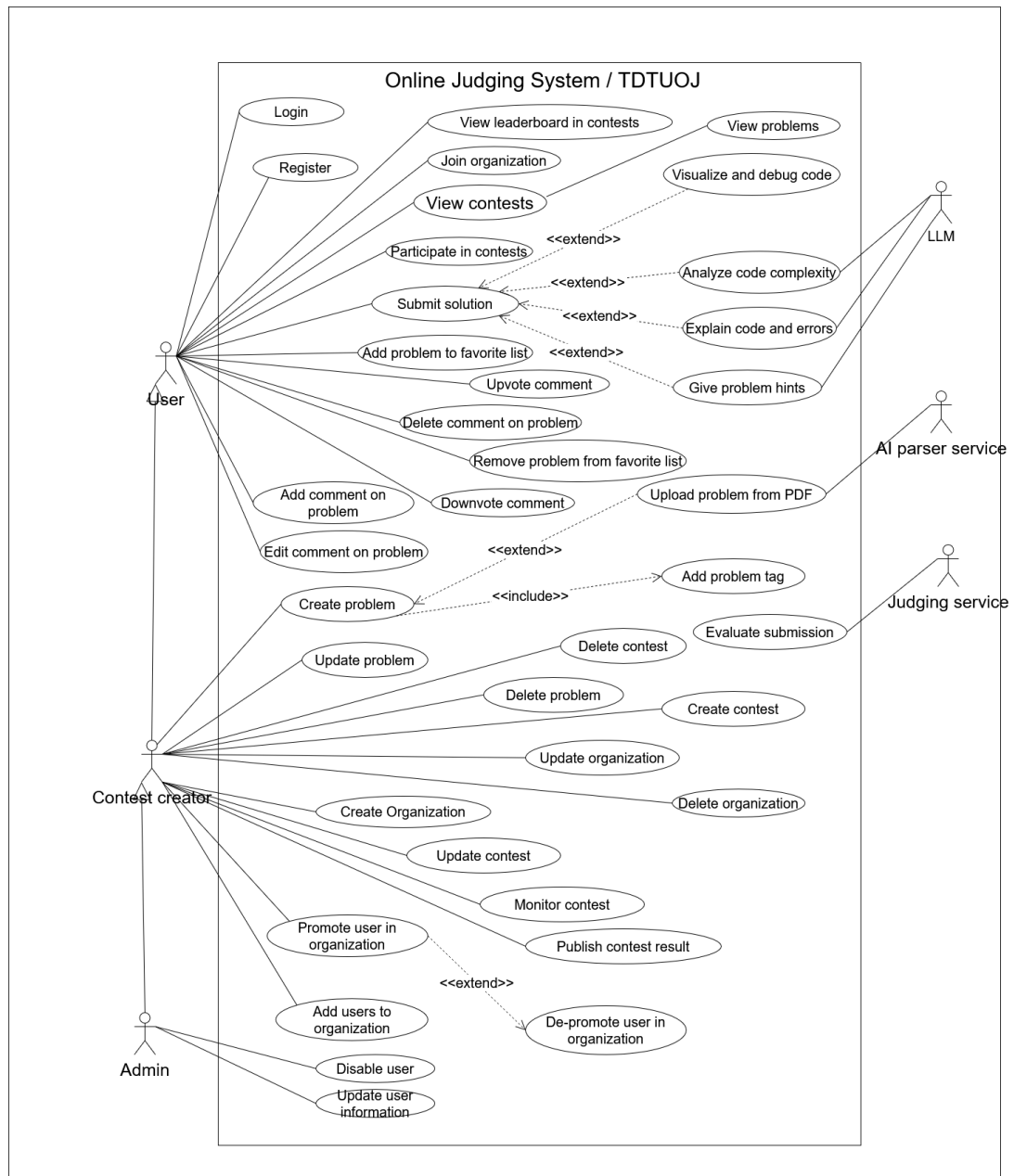
Mã UC	Tên Use Case	Mô tả
Nhóm 4: Quản lý và tham gia cuộc thi		
UC14	Tạo cuộc thi	Contest Creator hoặc Admin tạo một cuộc thi mới, cấu hình thông tin tổng quan, thời gian bắt đầu – kết thúc và danh sách bài toán.
UC15	Chỉnh sửa cuộc thi	Contest Creator hoặc Admin cập nhật thông tin của một cuộc thi, bao gồm điều chỉnh cấu hình hoặc thay thế bài toán trước khi cuộc thi bắt đầu.
UC16	Xóa cuộc thi	Contest Creator hoặc Admin xóa một cuộc thi chưa diễn ra khỏi hệ thống.
UC17	Đăng ký tham gia cuộc thi	Người dùng đăng ký tham gia một cuộc thi đang mở đăng ký. Hệ thống ghi nhận thông tin đăng ký và cấp quyền truy cập các bài toán trong cuộc thi khi đến thời gian quy định.
UC18	Tham gia cuộc thi	Người dùng đã đăng ký truy cập vào không gian thi cử, xem danh sách bài toán của cuộc thi và nộp bài trong khoảng thời gian cho phép.
UC19	Xem bảng xếp hạng	Người dùng xem bảng xếp hạng của một cuộc thi được cập nhật theo thời gian thực. Dữ liệu bảng xếp hạng được phục vụ từ cache Redis nhằm đảm bảo hiệu năng.
Nhóm 5: Hỗ trợ học tập bằng LLM		

(tiếp theo Bảng 3.2)

Mã UC	Tên Use Case	Mô tả
UC20	Gợi ý hướng giải	Người dùng yêu cầu gợi ý có định hướng cho một bài toán. LLM cung cấp gợi ý về hướng tiếp cận mà không tiết lộ lời giải hoàn chỉnh, nhằm kích thích tư duy giải quyết vấn đề của người học.
UC21	Phân tích độ phức tạp thuật toán	Người dùng gửi đoạn mã nguồn của lời giải để nhận phân tích độ phức tạp thời gian và không gian kèm theo giải thích chi tiết từ LLM.
UC22	Giải thích mã nguồn	Người dùng yêu cầu giải thích một đoạn mã nguồn. LLM phân tích logic, cấu trúc và ý nghĩa của từng phần trong đoạn mã, giúp người học hiểu rõ hơn về cách tiếp cận của lời giải.
UC23	Nhập đề bài từ PDF (AI Parser)	Contest Creator tải lên tệp đề bài dạng PDF. AI Parser Service phân tích tài liệu và tự động điền các trường thông tin bài toán, giúp rút ngắn thời gian soạn thảo nội dung.
Nhóm 6: Quản trị hệ thống (Admin)		
UC24	Xem danh sách người dùng	Admin xem toàn bộ danh sách tài khoản trong hệ thống, có thể tìm kiếm và lọc theo vai trò hoặc trạng thái tài khoản.
UC25	Xem chi tiết tài khoản	Admin xem thông tin chi tiết của một tài khoản người dùng bất kỳ.

(tiếp theo Bảng 3.2)

Mã UC	Tên Use Case	Mô tả
UC26	Tạo tài khoản	Admin tạo tài khoản mới cho người dùng và gán vai trò phù hợp ngay tại thời điểm tạo.
UC27	Cập nhật thông tin tài khoản	Admin chỉnh sửa thông tin hoặc thay đổi vai trò của một tài khoản người dùng bất kỳ.
UC28	Vô hiệu hóa tài khoản	Admin vô hiệu hóa tài khoản của một người dùng. Tài khoản bị vô hiệu hóa sẽ không thể đăng nhập vào hệ thống cho đến khi được khôi phục.



Hình 3.1: Sơ đồ Use Case tổng quan của hệ thống TDTUOJ

3.3.2 Các Use Case chính

UC01: Đăng ký tài khoản

UC02: Đăng nhập

UC03: Nộp bài và xem kết quả

UC04: Quản lý bài toán

UC05: Tạo và quản lý cuộc thi

UC06: Tham gia cuộc thi

UC07: Xem bảng xếp hạng

UC08: Hỗ trợ học tập bằng LLM

UC09: Nhập đề bài từ PDF (AI Parser)

UC10: Quản lý người dùng (Admin)

3.4 Yêu cầu phi chức năng (Non-functional Requirements)

CHƯƠNG 4. THIẾT KẾ HỆ THỐNG

4.1 Thiết kế kiến trúc

Hình 4.1: Kiến trúc tổng quan của hệ thống TDTUOJ

4.2 Thiết kế cơ sở dữ liệu / ERD

Hình 4.2: Sơ đồ thực thể – quan hệ của hệ thống TDTUOJ

4.2.1 Nhóm bảng quản lý người dùng

4.2.2 Nhóm bảng quản lý bài toán

4.2.3 Nhóm bảng quản lý cuộc thi

4.2.4 Nhóm bảng quản lý bài nộp

4.3 Thiết kế lớp

Hình 4.3: Sơ đồ lớp của hệ thống TDTUOJ

4.4 Thiết kế giao diện

4.4.1 Trang chủ

4.4.2 Trang danh sách bài toán

4.4.3 Trang chi tiết bài toán và nộp bài

4.4.4 Trang cuộc thi và bảng xếp hạng

4.4.5 Trang quản trị

CHƯƠNG 5. CÀI ĐẶT VÀ KIỂM THỬ

CHƯƠNG 6. KẾT LUẬN

TÀI LIỆU THAM KHẢO

(n.d.).

Espinoza, M. S., Goncalves, J., Leitao, P., Sanchez, J. L. G., & Herreros, A. (2012). Inverse kinematics of a 10 dof modular hyper-redundant robot resorting to exhaustive and error-optimization methods: A comparative study. *Robotics Symposium and Latin American Robotics Symposium (SBR-LARS), 2012 Brazilian*, 125–130.

Espinoza, M. S., Pereira, A. I., & Gonçalves, J. (2012). Optimization methods for hyper-redundant robots' inverse kinematics in biomedical applications. *AIP Conference Proceedings, 1479*, 818.

Tham, M. (2001, May). Writing research theses or dissertations [University of Newcastle Upon Tyne]. <http://lorien.ncl.ac.uk/ming/dept/Tips/writing/thesis/thesis-intro.htm>